Irys 노드 모니터링 및 메트릭 시스템

개요

Irys 노드의 안정적인 운영을 위해서는 체계적인 모니터링과 메트릭 수집이 필수입니다. 이 문서는 로그 설정, 성능 메트릭 수집, 알림 시스템 구성 방법을 상세히 설명합니다.

로깅 시스템

1. 로그 레벨 설정

Irys 노드는 Rust의 `tracing` 크레이트를 사용하여 구조화된 로깅을 제공합니다.

환경 변수를 통한 로그 레벨 설정

```
# 전체 로그 레벨 설정
export RUST_LOG=info

# 모듈별 세부 로그 레벨 설정
export RUST_LOG="irys_chain=debug,irys_consensus=info,irys_p2p=warn"

# 특정 크레이트만 디버그 모드
export RUST_LOG="irys_vdf=debug,warn"
```

사용 가능한 로그 레벨

1. **ERROR**: 심각한 오류만 기록
2. **WARN**: 경고 및 오류 기록
3. **INFO**: 일반 정보, 경고, 오류 기록 (기본값)
4. **DEBUG**: 디버그 정보 포함
5. **TRACE**: 모든 상세 정보 기록

2. 로그 출력 형식

기본 콘솔 출력

```
// 기본 설정 (main.rs에서)
let output_layer = tracing_subscriber::fmt::layer()
.with_line_number(true) // 라인 번호 포함
.with_ansi(true) // 컬러 출력
.with_file(true) // 파일명 포함
.with_writer(std::io::stdout);
```

JSON 형식 로그 (프로덕션 권장)

```
# JSON 형식으로 로그 출력
export RUST_LOG_FORMAT=json
```

예제 JSON 로그:

```
{
"timestamp": "2024-01-15T10:30:45.123Z",
```

```
"level": "INFO",
"target": "irys_consensus",
"module_path": "irys_consensus::vdf",
"file": "src/vdf/mod.rs",
"line": 142,
"message": "VDF step completed",
"fields": {
"step_number": 12345,
"duration_ms": 1200,
"difficulty": 1800000
}
}
```


3. 로그 파일 관리

로그 파일 위치

```
# 기본 로그 디렉토리
.irys/logs/
├── irys-node.log  # 메인 노드 로그
├── reth/  # Reth 실행 계층 로그
|  ├── reth.log
|  └── reth-debug.log
├── consensus.log  # 합의 관련 로그
```

```
├── p2p.log  # P2P 네트워킹 로그
└── storage.log  # 스토리지 관련 로그
```

로그 로테이션 설정

logrotate 설정 예제 (`/etc/logrotate.d/irys`):

```
/path/to/.irys/logs/*.log {
daily
rotate 30
compress
delaycompress
missingok
notifempty
create 644 irys irys
postrotate
systemctl reload irys-node
endscript
}
```

systemd 로그 관리

```
# systemd 서비스로 실행 시
journalctl -u irys-node -f  # 실시간 로그 확인
journalctl -u irys-node --since today  # 오늘 로그만
```

```
journalctl -u irys-node -n 100 # 최근 100줄
```

성능 메트릭 수집

1. 내장 메트릭

Irys 노드는 다음과 같은 핵심 메트릭을 제공합니다:

VDF 관련 메트릭

```
// VDF 성능 메트릭
pub struct VdfMetrics {
pub steps_per_second: f64, // 초당 VDF 단계 수
pub verification_time_ms: u64, // 검증 시간 (밀리초)
pub difficulty: u64, // 현재 난이도
pub reset_count: u64, // 리셋 횟수
pub parallel_threads: usize, // 병렬 스레드 수
}
```

블록체인 메트릭

```
pub struct ChainMetrics {
pub block_height: u64, // 현재 블록 높이
pub block_time_avg_ms: u64, // 평균 블록 시간
pub tx_pool_size: usize, // 트랜잭션 풀 크기
```

```
pub peer_count: usize, // 연결된 피어 수
pub sync_progress: f64, // 동기화 진행률 (0.0-1.0)
}
```

스토리지 메트릭

```
pub struct StorageMetrics {
pub total_chunks: u64, // 총 청크 수
pub storage_used_gb: f64, // 사용된 저장 공간 (GB)
pub storage_available_gb: f64, // 사용 가능한 저장 공간 (GB)
pub ingress_rate_mbps: f64, // 데이터 입력 속도 (Mbps)
pub packing_efficiency: f64, // 패킹 효율성 (0.0-1.0)
}
```

2. 메트릭 수집 방법

HTTP 엔드포인트를 통한 메트릭 조회

```
# 기본 노드 상태
curl http://localhost:8080/metrics

# VDF 상태
curl http://localhost:8080/metrics/vdf

# 스토리지 상태
```

```
curl http://localhost:8080/metrics/storage

# 네트워크 상태
curl http://localhost:8080/metrics/network
```

메트릭 응답 예제


```
{
"timestamp": "2024-01-15T10:30:45.123Z",
"node_id": "0x1234...",
"uptime_seconds": 86400,
"chain": {
"block_height": 12345,
"block_time_avg_ms": 12000,
"tx_pool_size": 42,
"peer_count": 8,
"sync_progress": 1.0
},
"vdf": {
"steps_per_second": 1500.0,
"verification_time_ms": 800,
"difficulty": 1800000,
"reset_count": 123,
```

```
"parallel_threads": 4
},
"storage": {
"total_chunks": 1000000,
"storage_used_gb": 256.5,
"storage_available_gb": 743.5,
"ingress_rate_mbps": 10.5,
"packing_efficiency": 0.85
}
}
```


3. Prometheus 통합

Prometheus 메트릭 엔드포인트

```
# Prometheus 형식 메트릭
curl http://localhost:8080/metrics/prometheus
```

메트릭 예제 (Prometheus 형식)

```
# HELP irys_block_height Current block height
# TYPE irys_block_height gauge
irys_block_height 12345

# HELP irys_vdf_steps_per_second VDF steps processed per second
```

```
# TYPE irys_vdf_steps_per_second gauge
irys_vdf_steps_per_second 1500.0

# HELP irys_storage_used_bytes Storage space used in bytes
# TYPE irys_storage_used_bytes gauge
irys_storage_used_bytes{module="0"} 275414777856
irys_storage_used_bytes{module="1"} 275414777856
irys_storage_used_bytes{module="2"} 275414777856

# HELP irys_peer_count Number of connected peers
# TYPE irys_peer_count gauge
irys_peer_count 8
```

Prometheus 설정 예제

```yaml
# prometheus.yml
global:
scrape_interval: 15s

scrape_configs:
- job_name: 'irys-node'
static_configs:
- targets: ['localhost:8080']
```

```
metrics_path: '/metrics/prometheus'

scrape_interval: 10s
```

모니터링 도구 통합

1. Grafana 대시보드

대시보드 구성 요소

노드 상태 패널:

- 노드 업타임
- 블록 높이
- 동기화 상태
- 피어 연결 수

VDF 성능 패널:

- VDF 단계 처리 속도
- 검증 시간 추이
- 난이도 변화
- 리셋 빈도

스토리지 패널:

- 저장 공간 사용률
- 데이터 입력 속도
- 패킹 효율성
- 청크 처리 통계

Grafana 쿼리 예제

```
# 블록 생성 속도 (분당)
rate(irys_block_height[5m]) * 60


# VDF 성능 추이
irys_vdf_steps_per_second


# 저장 공간 사용률
(irys_storage_used_bytes / (irys_storage_used_bytes + irys_storage_available_byt


# 피어 연결 안정성
avg_over_time(irys_peer_count[1h])
```

2. 알림 시스템 구성

Alertmanager 규칙

```yaml
# alerts.yml
groups:
- name: irys-node
rules:
# 노드 다운 알림
- alert: IrysNodeDown
expr: up{job="irys-node"} == 0
```

```yaml
    for: 1m
    labels:
    severity: critical
    annotations:
    summary: "Irys node is down"
    description: "Irys node has been down for more than 1 minute"

  # 동기화 지연 알림
  - alert: IrysSyncLagging
    expr: irys_sync_progress < 0.95
    for: 5m
    labels:
    severity: warning
    annotations:
    summary: "Irys node sync lagging"
    description: "Node sync progress is {{ $value }}"

  # VDF 성능 저하 알림
  - alert: IrysVdfSlowdown
    expr: irys_vdf_steps_per_second < 1000
    for: 2m
    labels:
    severity: warning
```

```
annotations:
summary: "VDF performance degraded"
description: "VDF processing at {{ $value }} steps/sec"


# 저장 공간 부족 알림
- alert: IrysStorageFull
expr: (irys_storage_used_bytes / (irys_storage_used_bytes + irys_storage_availab
for: 1m
labels:
severity: critical
annotations:
summary: "Storage space running low"
description: "Storage usage is {{ $value }}%"


# 피어 연결 부족 알림
- alert: IrysLowPeerCount
expr: irys_peer_count < 3
for: 5m
labels:
severity: warning
annotations:
summary: "Low peer count"
```

```
description: "Only {{ $value }} peers connected"
```

Slack 알림 설정

```
# alertmanager.yml
route:
group_by: ['alertname']
group_wait: 10s
group_interval: 10s
repeat_interval: 1h
receiver: 'slack-notifications'

receivers:
- name: 'slack-notifications'
slack_configs:
- api_url: 'YOUR_SLACK_WEBHOOK_URL'
channel: '#irys-alerts'
title: 'Irys Node Alert'
text: '{{ range .Alerts }}{{ .Annotations.summary }}: {{ .Annotations.descriptio
```

3. 커스텀 모니터링 스크립트

노드 상태 체크 스크립트

```bash
#!/bin/bash
# check_irys_node.sh

API_URL="http://localhost:8080"
ALERT_THRESHOLD_PEERS=3
ALERT_THRESHOLD_SYNC=0.95

# 노드 상태 확인
response=$(curl -s "$API_URL/metrics" || echo "ERROR")

if [ "$response" = "ERROR" ]; then
echo "CRITICAL: Node API not responding"
exit 2
fi

# JSON 파싱 (jq 필요)
peer_count=$(echo "$response" | jq -r '.chain.peer_count')
sync_progress=$(echo "$response" | jq -r '.chain.sync_progress')
block_height=$(echo "$response" | jq -r '.chain.block_height')

# 피어 수 확인
if [ "$peer_count" -lt "$ALERT_THRESHOLD_PEERS" ]; then
```

```
echo "WARNING: Low peer count ($peer_count)"
exit 1
fi

# 동기화 상태 확인
if (( $(echo "$sync_progress < $ALERT_THRESHOLD_SYNC" | bc -l) )); then
echo "WARNING: Sync lagging (${sync_progress})"
exit 1
fi

echo "OK: Node healthy - Height: $block_height, Peers: $peer_count, Sync: $sync
exit 0
```

자동 재시작 스크립트

```
#!/bin/bash
# auto_restart.sh

SERVICE_NAME="irys-node"
MAX_RESTART_ATTEMPTS=3
RESTART_DELAY=30

check_service() {
systemctl is-active --quiet "$SERVICE_NAME"
```

```
return $?
}

restart_service() {
echo "$(date): Restarting $SERVICE_NAME..."
systemctl restart "$SERVICE_NAME"
sleep "$RESTART_DELAY"
}

# 메인 로직
attempt=0
while [ $attempt -lt $MAX_RESTART_ATTEMPTS ]; do
if check_service; then
echo "$(date): Service is running normally"
exit 0
else
attempt=$((attempt + 1))
echo "$(date): Service down, attempt $attempt/$MAX_RESTART_ATTEMPTS"
restart_service
fi
done

echo "$(date): Failed to restart service after $MAX_RESTART_ATTEMPTS attempts"
```

```
exit 1
```

로그 분석 및 디버깅

1. 일반적인 로그 패턴

정상 작동 로그

```
2024-01-15T10:30:45.123Z INFO irys_consensus: VDF step completed step=12345 dura
2024-01-15T10:30:46.234Z INFO irys_p2p: New peer connected peer_id=0x1234... add
2024-01-15T10:30:47.345Z INFO irys_chain: Block produced height=12346 tx_count=5
```

경고 로그

```
2024-01-15T10:30:48.456Z WARN irys_p2p: Peer connection timeout peer_id=0x5678..
2024-01-15T10:30:49.567Z WARN irys_storage: Storage space low available_gb=50.2
2024-01-15T10:30:50.678Z WARN irys_vdf: VDF verification slow duration_ms=5000 e
```

오류 로그

```
2024-01-15T10:30:51.789Z ERROR irys_consensus: Block validation failed height=12
2024-01-15T10:30:52.890Z ERROR irys_storage: Chunk write failed chunk_id=0x9abc.
2024-01-15T10:30:53.901Z ERROR irys_p2p: Network partition detected connected_pe
```

2. 로그 분석 도구

grep을 이용한 로그 필터링

```
# 오류 로그만 추출
grep "ERROR" .irys/logs/irys-node.log

# 특정 시간대 로그
grep "2024-01-15T10:3[0-9]" .irys/logs/irys-node.log

# VDF 관련 로그만
grep "irys_vdf\|irys_consensus" .irys/logs/irys-node.log

# 성능 문제 관련 로그
grep -E "(slow|timeout|failed)" .irys/logs/irys-node.log
```

jq를 이용한 JSON 로그 분석

```
# JSON 로그에서 오류만 추출
cat .irys/logs/irys-node.log | jq 'select(.level == "ERROR")'

# 특정 모듈의 로그만
cat .irys/logs/irys-node.log | jq 'select(.target | startswith("irys_vdf"))'

# 성능 메트릭 추출
```

```
cat .irys/logs/irys-node.log | jq 'select(.fields.duration_ms > 2000)'
```

3. 성능 분석

시스템 리소스 모니터링

```
# CPU 사용률 모니터링
top -p $(pgrep irys-node)

# 메모리 사용량 확인
ps aux | grep irys-node

# 디스크 I/O 모니터링
iotop -p $(pgrep irys-node)

# 네트워크 연결 상태
netstat -tulpn | grep irys-node
```

벤치마크 및 프로파일링

```
# VDF 성능 테스트
irys-cli benchmark vdf --duration 60s

# 스토리지 성능 테스트
```

```
irys-cli benchmark storage --size 1GB


# 네트워크 지연 측정

irys-cli benchmark network --peers 5
```

이 모니터링 가이드를 통해 Irys 노드의 상태를 실시간으로 추적하고, 문제 발생 시 신속하게 대응할 수 있습니다. 정기적인 메트릭 검토와 알림 시스템 구성을 통해 안정적인 노드 운영을 보장하시기 바랍니다.